

UNIwersYTET RZESZOWSKI
WYDZIAŁ NAUK ŚCISŁYCH I TECHNICZNYCH
INSTYTUT INFORMATYKI



Łukasz Warzocha, Mateusz Serafin, Przemysław Sulowski
134983, 134969, 134975

Informatyka

Implementacja algorytmów SI w grze "Czołgi 2D"

Praca projektowa

Praca wykonana pod kierunkiem
mgr inż. Marcin Mrukowicz

Rzeszów 2026

Spis treści

1. Opis założeń projektu	6
1.1. Cel projektu	6
1.2. Założenia technologiczne	6
1.3. Wykorzystanie AI w przygotowywaniu projektu	6
2. Opis struktury projektu	8
2.1. Technologie wykorzystane w projekcie	8
2.2. Struktura plików źródłowych	8
2.3. Tryb Deathmatch - algorytm Minimax z przycinaniem alfa-beta	9
2.3.1. Idea algorytmu Minimax.....	9
2.3.2. Funkcja oceny pozycji	9
2.3.3. Przycinanie alfa-beta.....	10
2.3.4. Implementacja w pliku enemy.py	10
2.4. Tryb Capture Point - algorytm A*.....	11
2.4.1. Idea algorytmu A*	11
2.4.2. Heurystyka euklidesowa i wagi przejść kafelków	12
2.4.3. Cache'owanie ścieżki i warunki przeliczania	12
2.4.4. Implementacja w pliku enemy2.py	12
2.5. Heurystyki uzupełniające	13
2.6. Napotkane problemy i sposoby ich rozwiązania	14
3. Prezentacja warstwy użytkowej projektu.....	16
3.1. Menu główne i ekran wyboru trybu	16
3.2. Ekran rozgrywki - tryb Deathmatch	16
3.3. Ekran rozgrywki - tryb Capture Point	18
3.4. System power-upów	19
3.5. Przykładowe scenariusze rozgrywki	20
3.6. Instrukcja uruchomienia aplikacji	20
4. Podsumowanie	21
4.1. Wnioski z realizacji projektu	21
4.2. Możliwości rozszerzenia projektu	21
Spis rysunków	23
Spis tabel	24
Spis listingów	25

1. Opis założeń projektu

1.1. Cel projektu

Głównym celem projektu była praktyczna implementacja dwóch klasycznych algorytmów sztucznej inteligencji. Wybrane algorytmy - Minimax z przycinaniem alfa-beta oraz A*. Ich zastosowanie poza laboratoryjnymi warunkami szachownicy wymaga rozwiązania szeregu problemów: ograniczeń czasowych, zachowań kolizyjnych, synchronizacji wielu agentów oraz balansowania między optymalnością decyzji a kosztem obliczeniowym.

Aby zapewnić, że oba algorytmy będą prezentowane w odmiennym kontekście, gra została zaprojektowana z dwoma niezależnymi trybami rozgrywki. W trybie Deathmatch przeciwnik wykorzystuje algorytm Minimax do podejmowania decyzji o ruchu w konfrontacji z graczem, natomiast w trybie Capture Point korzysta z algorytmu A* do wyznaczenia ścieżki do dynamicznie zmieniającej się strefy przejmowania.

Dodatkowym celem było zaprojektowanie systemu, w którym oba algorytmy współpracują z heurystykami uzupełniającymi (wykrywanie linii strzału, unikanie friendly fire, obsługa kolizji), tworząc wiarygodne zachowanie przeciwnika.

1.2. Założenia technologiczne

Projekt zrealizowano w języku Python z wykorzystaniem biblioteki pygame, która stanowi silnik graficzny i zdarzeniowy gry. Wybór języka podyktowany został jego przejrzystą składnią matematyczną (kluczową przy implementacji algorytmów SI).

Pełna lista wykorzystanych bibliotek obejmuje:

- `pygame` - silnik gry (okno, renderowanie, kolizje, klawiatura, dźwięk)
- `random` - losowanie map, pozycji spawnu, typów power-upów oraz wyborów AI
- `heapq` - kolejka priorytetowa wykorzystywana w algorytmie A*
- `glob` - wyszukiwanie plików map w folderach Maps/ i Maps2/

Mapa gry ma stałe wymiary 800x600 pikseli i jest podzielona na siatkę kafelków 20x15 o rozmiarze 40x40 pikseli każdy. Czołg gracza ma hitbox 20x20 pikseli i porusza się z prędkością 2,7 piksela na klatkę, natomiast czołgi przeciwników poruszają się wolniej - z prędkością 2 piksele na klatkę. Różnica jest celowa: daje graczowi przewagę manewrową, kompensując fakt, że każdy bot zna całą mapę i planuje optymalnie.

1.3. Wykorzystanie AI w przygotowywaniu projektu

W trakcie realizacji projektu korzystano z modelu językowego (Claude) w następujących obszarach:

- **Wsparcie przy implementacji** - konsultacja struktury klas, generowanie szablonów funkcji pomocniczych.
- **Debugowanie** - diagnoza nietrywialnych błędów, m.in. botów „kręcących się” wokół własnej osi (problem wynikał z rotacji lufy co klatkę przed właściwym strzałem) oraz strzałów przelatujących przez zniszczalne ściany.

- **Refaktoryzacja kodu** - restrukturyzacja funkcji `_minimax` celem lepszej czytelności i weryfikowalności poprawności algorytmu.

Każdy wygenerowany przez AI fragment kodu został zweryfikowany poprzez (uruchamianie gry i obserwacja zachowania AI w różnych sytuacjach) oraz analizę logiczną. Żadne dane zewnętrzne nie były wykorzystane w projekcie.

W projekcie nie wykorzystano generatywnej AI do tworzenia grafik, animacji ani plansz, wszystkie elementy graficzne (czołg, pocisk, power-upy, ściany) są prostymi figurami geometrycznymi renderowanymi przez pygame bezpośrednio z kodu.

2. Opis struktury projektu

2.1. Technologie wykorzystane w projekcie

Projekt został zaimplementowany w Pythonie w wersji 3.12 z wykorzystaniem biblioteki pygame 2.6.1. Środowiskiem programistycznym był PyCharm Community Edition. Repozytorium kodu jest organizowane w jeden katalog roboczy.

Architektura gry opiera się na pętli zdarzeń pygame z zachowaniem stałej liczby klatek (60 FPS). Każda klatka obejmuje:

1. Obsługę zdarzeń klawiatury (input gracza)
2. Aktualizację AI przeciwników
3. Aktualizację pozycji pocisków i detekcję kolizji
4. Renderowanie sceny na ekranie

2.2. Struktura plików źródłowych

Projekt składa się z następujących plików źródłowych:

- `main.py` - punkt wejścia aplikacji; obsługa menu, wybór trybu, pętla gry, zarządzanie stanami rozgrywki, spawnowanie power-upów i przeciwników
- `tank.py` - klasa bazowa Tank reprezentująca czołg (gracza lub przeciwnika); obsługa ruchu, rotacji, strzelania, kolizji, power-upów, systemu zdrowia
- `enemy.py` - klasa EnemyTank dziedzicząca po Tank; implementacja AI opartej na Minimax z przycinaniem alfa-beta dla trybu Deathmatch
- `enemy2.py` - klasa EnemyTankCP dziedzicząca po Tank; implementacja AI opartej na A* dla trybu Capture Point
- `bullet.py` - klasa Bullet reprezentująca pocisk; obsługa ruchu, animacji, detekcji wyjścia poza ekran
- `walls.py` - klasy ścian niszczalnych Wall oraz niezniszczalnych IndestructibleWall
- `powerup.py` - klasa PowerUp reprezentująca bonus na mapie (życie, pancerz, silny strzał)
- `map_loader.py` - funkcje wczytujące mapy z plików tekstowych .txt z folderów Maps (Deathmatch) i Maps2 (Capture Point)

Mapa jest zapisana w pliku tekstowym każdy znak reprezentuje jeden kafelek 40x40 pikseli, # to ściana niezniszczalna, X to ściana niszczalna, spacja to wolne pole. Pozwala to na łatwe dodawanie nowych map bez modyfikacji kodu.

2.3. Tryb Deathmatch - algorytm Minimax z przycinaniem alfa-beta

2.3.1. Idea algorytmu Minimax

Minimax jest klasycznym algorytmem decyzyjnym przeznaczonym dla gier dwuosobowych z pełną informacją i ruchem naprzemiennym. Algorytm zakłada, że obaj gracze grają optymalnie: jeden (MAX) dąży do maksymalizacji oceny pozycji, drugi (MIN) do jej minimalizacji.

Drzewo Minimax o głębokości d reprezentuje sekwencję d kolejnych ruchów. Korzeń drzewa odpowiada bieżącej pozycji, każdy poziom poniżej reprezentuje jeden hipotetyczny ruch, na przemian bota (warstwa MAX) i gracza (warstwa MIN). W liściach drzewa (głębokość 0) wywoływana jest funkcja oceny pozycji $eval(s)$, której wartość propaguje się w górę drzewa zgodnie z regułą:

$$\text{Minimax}(s) = \begin{cases} eval(s) & \text{gdy } d = 0 \\ \max_{s' \in \text{succ}(s)} \text{Minimax}(s') & \text{gdy gracz to MAX} \\ \min_{s' \in \text{succ}(s)} \text{Minimax}(s') & \text{gdy gracz to MIN} \end{cases} \quad (2.1)$$

W zaimplementowanym projekcie głębokość drzewa wynosi $d=4$, czyli bot przewiduje dwa swoje ruchy i dwa odpowiedzi gracza. Strategia jest z natury defensywna - bot wybiera ruch, który jest dla niego najlepszy nawet w najgorszym scenariuszu, co eliminuje błędy typu 'wskoczę na linię ognia, bo akurat gracz patrzy w drugą stronę'.

2.3.2. Funkcja oceny pozycji

Funkcja oceny $_eval$ składa się z czterech komponentów połączonych liniowo:

$$f(b, p, \text{cost}, \text{pu}) = -2 \cdot d_{Ch}(b, p) + B_{LOS} + B_{PU} - 2 \cdot \text{cost}_{acc} \quad (2.2)$$

gdzie:

Odległość Czebyszewa do gracza z wagą -2 :

$$d_{Ch}(a, b) = \max(|a_x - b_x|, |a_y - b_y|) \quad (2.3)$$

Dystans Czebyszewa jest lepszą miarą niż odległość Manhattan dla ruchu w siatce 4-kierunkowej, ponieważ uwzględnia naturalne obwiednie ruchów (czołg może poruszać się w osi X i Y niezależnie). Im bliżej gracza, tym wyższa ocena, co motywuje bota do aktywnego ścigania przeciwnika.

Bonus linii strzału (LOS) $B_{LOS} = +80$ przyznawany, gdy bot i gracz znajdują się w tej samej osi (z tolerancją 16 pikseli) oraz między nimi nie ma niezniszczalnej ściany blokującej tor pocisku. Jest to bardzo silny sygnał: pozycja umożliwia czysty strzał jest mocno premiowana.

Bonus power-upa $B_{PU} \in [0, 40]$ przyznawany, gdy gracz jest daleko ($d_{Ch}(b, p) > 120$ px) a najbliższy power-up blisko ($d_{Ch}(b, \text{pu}) < 100$ px). Wartość bonusu skaluje się odwrotnie z odległością: $B_{PU} = \max(0, 40 - d_{Ch}(b, \text{pu})/3)$. Pozwala botowi zbierać użyteczne bonusy w momentach, gdy gonienie gracza nie ma sensu.

Kara za koszt drogi $-2 \cdot \text{cost}_{acc}$, gdzie cost_{acc} to suma kosztów przejść przez zniszczalne ściany na symulowanej trasie. Każda zniszczalna ściana to koszt $\text{COST_DEST} = 8$. Komponent ten zniechęca bota do planowania ruchów wymagających niszczenia ścian, jeśli istnieje alternatywna trasa.

Dodatkowo, w turze gracza w symulacji uwzględniony jest szczegół: jeśli na danej hipotetycznej pozycji gracz miałby linię strzału do bota, do oceny tej pozycji dodawana jest kara -60 . Symulacja 'gracz strzela

zamiast uciekać' wymusza na Minimaxie aktywne unikanie pozycji wystawiających bota na ostrzał, nie tylko maksymalizację bliskości.

2.3.3. Przycinanie alfa-beta

Przycinanie alfa-beta jest optymalizacją algorytmu Minimax pozwalającą pominąć całe gałęzie drzewa, które nie wpłyną na ostateczną decyzję. Algorytm utrzymuje dwie wartości graniczne:

- α - najlepsza (najwyższa) ocena, jaką MAX może już na pewno zagwarantować w przeszukanej części drzewa
- β - najlepsza (najniższa) ocena, jaką MIN może już na pewno zagwarantować

Gdy w trakcie przeszukiwania zachodzi warunek $\beta \leq \alpha$, dalsze rozwijanie tej gałęzi nie ma sensu - jest oczywiste, że rodzic czyli przeciwnik nigdy nie wybierze tej ścieżki, ponieważ jego dotychczasowo poznana opcja jest co najmniej tak dobra jak obecnie analizowana. Przycinanie nie zmienia ostatecznie wybranego ruchu tylko skraca obliczenia.

Pełne drzewo Minimax o głębokości $d=4$ i rozgałęzieniu 4 (cztery możliwe kierunki ruchu) miałoby teoretycznie $4^4 = 256$ liści. Po zastosowaniu przycinania alfa-beta liczba realnie ocenianych węzłów spada do rzędu $\mathcal{O}(b^{d/2})$ (w optymalnym przypadku uporządkowania ruchów), co w praktyce pozwala na uruchamianie Minimax płynnie w czasie rzeczywistym.

2.3.4. Implementacja w pliku enemy.py

Klasa EnemyTank dziedziczy po klasie bazowej Tank i rozszerza ją o logikę decyzyjną opartą na Minimax.

Listing 2.1. Implementacja Minimax z przycinaniem alfa-beta (fragment z enemy.py)

```
def _minimax(bx, by, px, py, indest_fs, dest_fs, cost_acc,
            depth, alpha, beta, maxi, powerups=None):
    if depth == 0:
        return _eval(bx, by, px, py, indest_fs, powerups, cost_acc)

    if maxi: # Tura bota (MAX)
        best = -INF
        for d in DIRS:
            nx, ny = bx + d[0]*STEP, by + d[1]*STEP
            c = _move_cost(nx, ny, indest_fs, dest_fs)
            if c == INF:
                continue
            val = _minimax(nx, ny, px, py, indest_fs, dest_fs,
                          cost_acc + c, depth-1, alpha, beta, False, powerups)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                break # PRZYCINANIE alfa-beta
        return best
    else: # Tura gracza (MIN)
        if _player_can_shoot(bx, by, px, py, indest_fs):
            return _eval(...) - 60 # kara za strzał gracza
        best = INF
        for d in DIRS:
            nx, ny = px + d[0]*STEP, py + d[1]*STEP
```



```

# (walidacja kafelka...)
val = _minimax(bx, by, nx, ny, indest_fs, dest_fs,
               cost_acc, depth-1, alpha, beta, True, powerups)
best = min(best, val)
beta = min(beta, best)
if beta <= alpha:
    break
return best

```

Ze względu na koszt obliczeniowy Minimax (rzędu kilkuset wywołań `_eval` na jedno wywołanie `_best_direction`), uruchomienie algorytmu w każdej klatce gry powodowałoby spadki wydajności. Zastosowano więc dwie optymalizacje czasowe:

- **Cykl decyzyjny** - Minimax uruchamiany jest co `THINK EVERY = 15` klatek, a przez pozostałe 14 klatek bot kontynuuje ruch w ostatnio wybranym kierunku. Daje to efektywny krok decyzyjny, wystarczająco szybki dla gracza.
- **Losowy offset** - każdy bot ma indywidualny losowy offset czasowy (`random.randint(0, THINK EVERY-1)`), więc boty nie myślą wszystkie w tej samej klatce.

Dodatkowo wykryto, że przy długiej grze bot może 'utknąć' w patowych konfiguracjach (np. między dwoma ścianami z Minimax'em wskazującym kierunek niemożliwy do zrealizowania). Zaimplementowano detekcję stanu utknięcia: jeśli pozycja bota nie zmieniła się przez 30 klatek, wymuszany jest wybór losowego wolnego kierunku.

Po wybraniu nowego kierunku przez Minimax, zanim bot zacznie się w nim poruszać, wykonywane jest tzw. autocentrowanie do osi kafelka prostopadłej do kierunku ruchu: przy skręcie w poziom wyrównywana jest pozycja pionowa Y bota do środka kafelka, przy skręcie w pion - pozycja pozioma X. Mechanizm działa z tolerancją 12 pikseli i zapobiega dryfowi między kafelkami, który mógłby spowodować, że bot wjedzie w konfigurację, z której nie da się ruszyć dalej w wybranym kierunku.

Niezależnie od powyższego, w sytuacji gdy bot mimo wszystko wjedzie w ścianę, stosowany jest drugi mechanizm awaryjny - snap do środka kafelka po kolizji ze ścianą.

2.4. Tryb Capture Point - algorytm A*

2.4.1. Idea algorytmu A*

A* jest algorytmem wyszukiwania optymalnej ścieżki w grafie ważonym, łączącym zalety algorytmu Dijkstry (gwarancja optymalności) z heurystycznym przeszukiwaniem 'najlepszy najpierw' (efektywność czasowa).

W zaimplementowanym wariancie graf jest dyskretną siatką kafelków 20x15 (300 węzłów), a każdy węzeł jest połączony z czterema sąsiadami (góra, dół, lewo, prawo). Algorytm utrzymuje dwie metryki dla każdego rozważanego węzła n:

- $g(n)$ - rzeczywisty koszt dotarcia od węzła startowego do n
- $h(n)$ - heurystyczny szacunek pozostałego kosztu od n do celu

Priorytet rozwijania węzła to $f(n) = g(n) + h(n)$. Kolejka priorytetowa zwraca zawsze węzeł o najniższym f , dzięki czemu algorytm szybko zbiega do celu, jednocześnie gwarantując optymalność ścieżki pod warunkiem, że heurystyka h jest dopuszczalna (nigdy nie przeszacowuje rzeczywistego kosztu).

2.4.2. Heurystyka euklidesowa i wagi przejść kafelków

Wykorzystaną heurystyką jest dystans euklidesowy:

$$h(a, b) = \sqrt{(a_x - b_x)^2 + (a_y - b_y)^2} \quad (2.4)$$

Euklidesowa odległość po prostej między dwoma punktami nie może być większa niż rzeczywista trasa po siatce 4-kierunkowej, ponieważ trasa po siatce zawsze realizuje przemieszczenie w osiach X i Y oddzielnie (sekwencja kroków w poziomie + kroków w pionie), a suma długości boków trójkąta prostokątnego jest co najmniej równa przeciwprostokątnej.

Koszt przejścia step cost zależy od typu kafelka:

Tabela 2.1. Wagi przejść kafelków w algorytmie A*

Typ kafelka	Koszt	Interpretacja
Wolne pole	1	przejście natychmiastowe
Zniszczalna ściana	6	wymaga rozwalenia (czas + pocisk)
Niezniszczalna ściana	∞	kafelek niedostępny

Wartość 6 dla zniszczalnej ściany jest dobrana z rozmysłem: bot wybierze obejście wolnymi polami, dopóki obejście wymaga mniej niż 6 dodatkowych kafelków. Gdy obejście byłoby znacznie dłuższe, opłaca się rozwalić ścianę. Naturalnie tworzy to zachowanie typu 'rozwałam ścianę tylko, gdy to się opłaca'.

2.4.3. Cache'owanie ścieżki i warunki przeliczania

Algorytm A* nie jest uruchamiany co klatkę - byłoby to zbyt kosztowne i niepotrzebne. Obliczona ścieżka jest przechowywana w polu `self.path` jako lista współrzędnych kafelków i przeliczana tylko wtedy, gdy:

- Zmienił się cel (punkt przeniósł się do innego sektora po przekroczeniu progu 25% postępu)
- Zmienił się układ niezniszczalnych ścian (zniszczona ściana otwiera nowe trasy)
- Lista ścieżki została wyczerpana (bot dotarł do kolejnego węzła końcowego)

Po dotarciu do każdego kafelka ścieżki ten kafelek jest usuwany z `self.path`, a bot kieruje się do następnego. W ten sposób, mimo iż A* jest kosztowny obliczeniowo, jego średnie obciążenie procesora jest niskie (typowo kilka wywołań na sekundę).

2.4.4. Implementacja w pliku `enemy2.py`

Listing poniżej przedstawia główną funkcję implementującą algorytm A*.

Listing 2.2. Implementacja algorytmu A* (fragment z `enemy2.py`)

```
def _a_star(self, start, goal, indest_walls, dest_walls_coords):
    # Heurystyka: odleglosc Euklidesowa
    def h(a, b):
        return ((a[0]-b[0])**2 + (a[1]-b[1])**2) ** 0.5

    queue = [(0, start)]
    came_from = {start: None}
    cost_so_far = {start: 0}
```

```

while queue:
    _, curr = heapq.heappop(queue)
    if curr == goal:
        break
    for d in [(0,1), (0,-1), (1,0), (-1,0)]:
        nxt = (curr[0]+d[0], curr[1]+d[1])
        if not (0 <= nxt[0] < 20 and 0 <= nxt[1] < 15):
            continue
        if nxt in indest_walls:
            continue # niezniszczalna - pomijamy
        step = 6 if nxt in dest_walls_coords else 1
        new_cost = cost_so_far[curr] + step
        if nxt not in cost_so_far or new_cost < cost_so_far[nxt]:
            cost_so_far[nxt] = new_cost
            heapq.heappush(queue, (new_cost + h(nxt, goal), nxt))
            came_from[nxt] = curr

# Odtwarzamy ścieżkę wstecz od celu do startu
path, c = [], goal
while c in came_from and c != start:
    path.append(c)
    c = came_from[c]
return path[::-1]

```

Strukturą danych przechowującą węzły do rozwinięcia jest kolejka priorytetowa zaimplementowana przez moduł `heapq` z biblioteki standardowej Pythona. Słownik `came_from` zapamiętuje rodzica każdego węzła, co umożliwia rekonstrukcję ścieżki przez cofanie się od węzła docelowego do startowego po zakończeniu przeszukiwania.

Zachowanie bota wewnątrz strefy przejmowania nie wykorzystuje już A^* - gdy bot dotrze do strefy, losuje losowy punkt patrolu wewnątrz strefy i jedzie do niego. Jeśli gracz również znajduje się w strefie, celem patrolu staje się sam gracz (bot przechodzi w tryb agresywny). Po dotarciu do celu losowany jest nowy punkt, aby bot nie stał w miejscu i nie był łatwym celem.

Dodatkowo, gdy bot jest jeszcze w drodze do strefy (tryb `_drive_to_point`), uwzględniana jest obecność power-upów na mapie. Jeśli najbliższy power-up znajduje się bliżej bota (w metryce Manhattan) niż środek strefy przejmowania, bot tymczasowo zmienia cel A^* na okolicę tego power-upa (jego `rect` rozszerzony o 60 pikseli w każdą stronę). Pozwala to botowi zbierać bonusy po drodze bez rezygnacji z głównego celu, ale nie przerywa już rozpoczętego przejmowania - logika ta działa wyłącznie poza strefą.

2.5. Heurystyki uzupełniające

Wykrywanie linii strzału

Wykrywanie linii strzału (Line Of Sight, LOS) wykorzystywane jest w dwóch miejscach algorytmu z różnymi stopniami restrykcyjności.

W funkcji oceny Minimax `_eval` sprawdzenie LOS odbywa się przez rzucenie wirtualnego promienia ze stałym krokiem `STEP=20` pikseli od pozycji bota do pozycji gracza. Promień blokowany jest wyłącznie przez niezniszczalne ściany - zniszczalne traktowane są jak potencjalna 'zasłona dymna' (bot może je rozwalić strzałem, więc obecność takiej ściany nie jest sygnałem braku LOS).

Natomiast w rzeczywistej decyzji o oddaniu strzału `_can_shoot`, sprawdzenie jest ściślejsze: promień rzucany jest ze mniejszym krokiem 10 pikseli i blokowany przez obie grupy ścian (zniszczalne i niezniszczalne). Logika ta zapobiega marnowaniu pocisków, które rozwaląby ścianę, ale nie trafiłyby w gracza.

Unikanie friendly fire i marnowania pocisków

Zaimplementowano dwa mechanizmy zapobiegające bezsensownym strzałom:

- Friendly fire avoidance - przed oddaniem strzału bot sprawdza, czy w jego linii ognia nie stoi inny bot (sojusznik). Funkcja `_ally_in_line` sprawdza wszystkich sojuszników w zasięgu 200 pikseli na osi kierunku ruchu, z tolerancją prostopadłą 22 pikseli (czołg ma 20px szerokości, dodatkowa tolerancja zabezpiecza przed strzałem w bok sojusznika).
- Friendly bullet detection - przed oddaniem strzału w zniszczalną ścianę bot sprawdza, czy w tym samym kierunku nie leci już pocisk z innego źródła (innego bota lub gracza). Funkcja `_friendly_bullet_in_path` szuka pocisków o tym samym kierunku w zasięgu 70 pikseli przed botem. Mechanizm ten zapobiega sytuacji, w której bot strzela w ścianę, którą za moment zniszczy już lecący pocisk - jego własny pocisk poleciałby wtedy do końca ekranu w pustą przestrzeń.

Zachowanie kolizyjne

System obsługi kolizji obejmuje dwa nieoczywiste mechanizmy:

- Snap do środka kafelka - gdy bot po zniszczalnej kolizji wciąż znajduje się wewnątrz ściany (np. wskutek faktu, że jego hitbox po rotacji ma rozmiar 34x34 pikseli a nie 20x20, pozycja bota jest wyciągana do środka aktualnego kafelka, czyli wielokrotności 40 plus 20). Środek kafelka to jedyny punkt w siatce, w którym 34-pikselowy hitbox ma gwarancję zmieszczenia się w 40-pikselowym kafelku bez stykania się ze ścianami sąsiednich kafelków.
- Unik od sojusznika - gdy bot wpadnie na innego bota, cofa się do pozycji sprzed ruchu i wybiera deterministycznie kierunek prostopadły do osi sojusznika (jeśli zderzenie nastąpiło w osi X, ucieka w osi Y, i odwrotnie). Wybór jest deterministyczny, a nie losowy, oraz ma cooldown 20 klatek - bez niego boty wpadające na siebie kręciły się wokół własnej osi co klatkę.

2.6. Napotkane problemy i sposoby ich rozwiązania

W trakcie implementacji projektu napotkano i rozwiązano szereg problemów. Najważniejsze z nich opisano poniżej.

Problem 2: Boty utykały w ścianach po rotacji.

W pliku `tank.py` obraz czołgu `base_image` jest tworzony jako powierzchnia 34x34 piksele - musi pomieścić korpus 22x22, gąsienice wystające po bokach oraz lufę wystającą ku górze. Jednocześnie w konstruktorze klasy `Tank` hitbox kolizyjny `self.rect` jest jawnie ustawiony na 20x20 pikseli. Problem pojawia się przy pierwszym wywołaniu metody `rotate`, która zastępuje hitbox bounding-boxem obrazu: `self.rect = self.image.get_rect(...)`. W efekcie hitbox bota po pierwszym obrocie urasta do 34x34 pikseli (rozmiaru obrazu) i nie wraca już do pierwotnego rozmiaru. W konsekwencji bot mógł wjechać w konfigurację, z której nie dało się wyjść standardowym snapem do rogu kafelka, ponieważ 34-pikselowy hitbox w rogu zawsze styka się ze ścianami sąsiednich kafelków.

Rozwiązanie: snap do środka kafelka (współrzędne typu $20 + 40k$), a nie do rogu - tylko środek kafelka gwarantuje, że 34-pikselowy hitbox zmieści się w 40-pikselowym kafelku bez stykania się ze ścianami sąsiednich kafelków (margines 3 piksele z każdej strony).

Problem 3: Boty kręciły się wokół własnej osi.

Bot wykonywał obrót lufy do gracza w każdej klatce, w której Minimax nie był uruchamiany (przez 14 klatek z 15-klatkowego cyklu), niezależnie od tego, czy reload pocisku był gotowy. Powodowało to migotanie wizualne i obrazanie się bota.

Rozwiązanie: rotacja lufy wykonywana tylko, gdy `get_reload_progress() >= 1.0` (czyli pocisk jest gotowy do strzału).

Problem 5: Boty wpadające na siebie kręciły się.

Gdy bot zderzył się z innym botem, każda kolejna klatka kolizji wybierała losowy nowy kierunek z listy preferowanych. Skutkiem był efekt nieskończonej rotacji bez rzeczywistego ruchu.

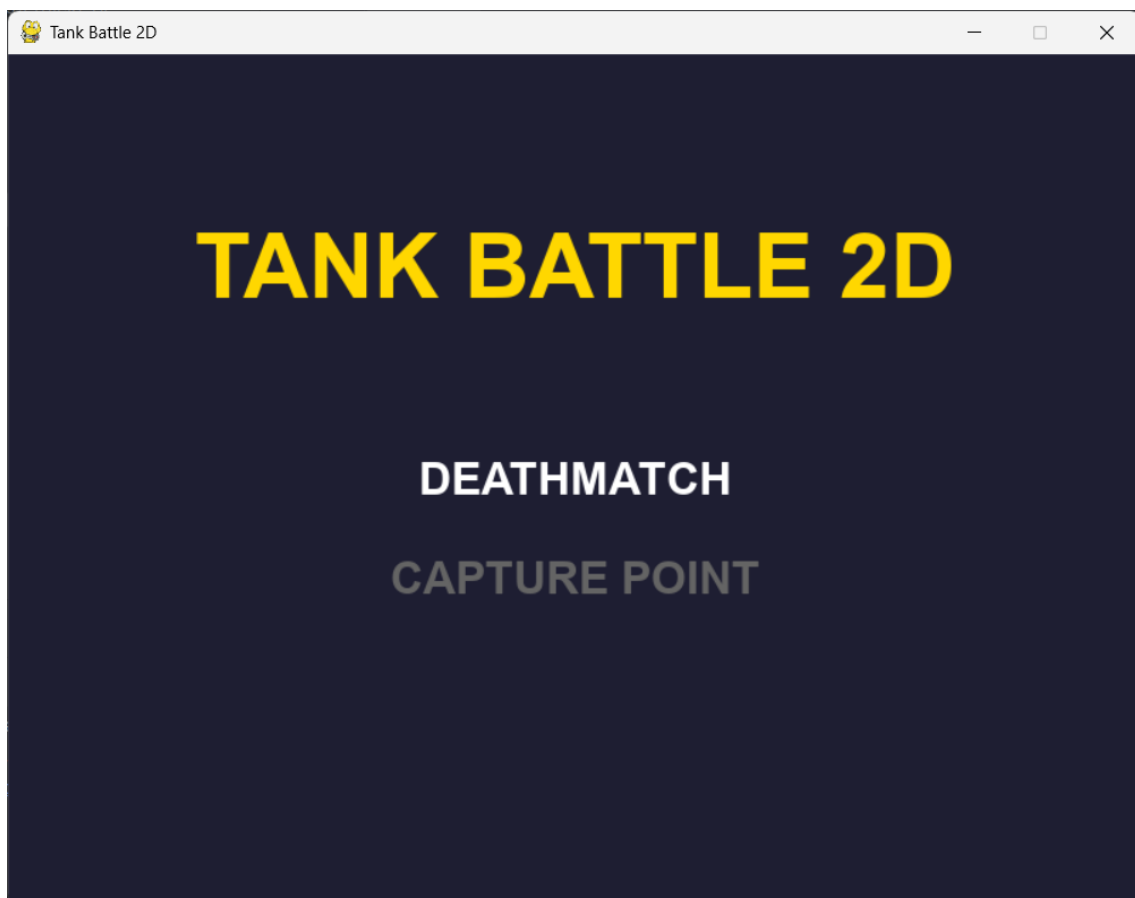
Rozwiązanie: deterministyczny wybór kierunku (zawsze ten sam dla danego ustawienia botów) plus cooldown 20 klatek przed kolejną zmianą kierunku.

3. Prezentacja warstwy użytkowej projektu

3.1. Menu główne i ekran wyboru trybu

Po uruchomieniu aplikacji wyświetlane jest menu główne z wyborem trybu rozgrywki Deathmatch lub Capture Point. Nawigacja odbywa się klawiszami strzałek góra/dół, zatwierdzenie wyboru klawiszem Enter.

Dla trybu Deathmatch wybór trybu jest następnie poprzedzany ekranem ustawień, na którym gracz wybiera liczbę przeciwników (1-3). Tryb Capture Point uruchamia rozgrywkę bezpośrednio po wyborze, ponieważ jego struktura zakłada dynamicznie zmieniającą się liczbę przeciwników.



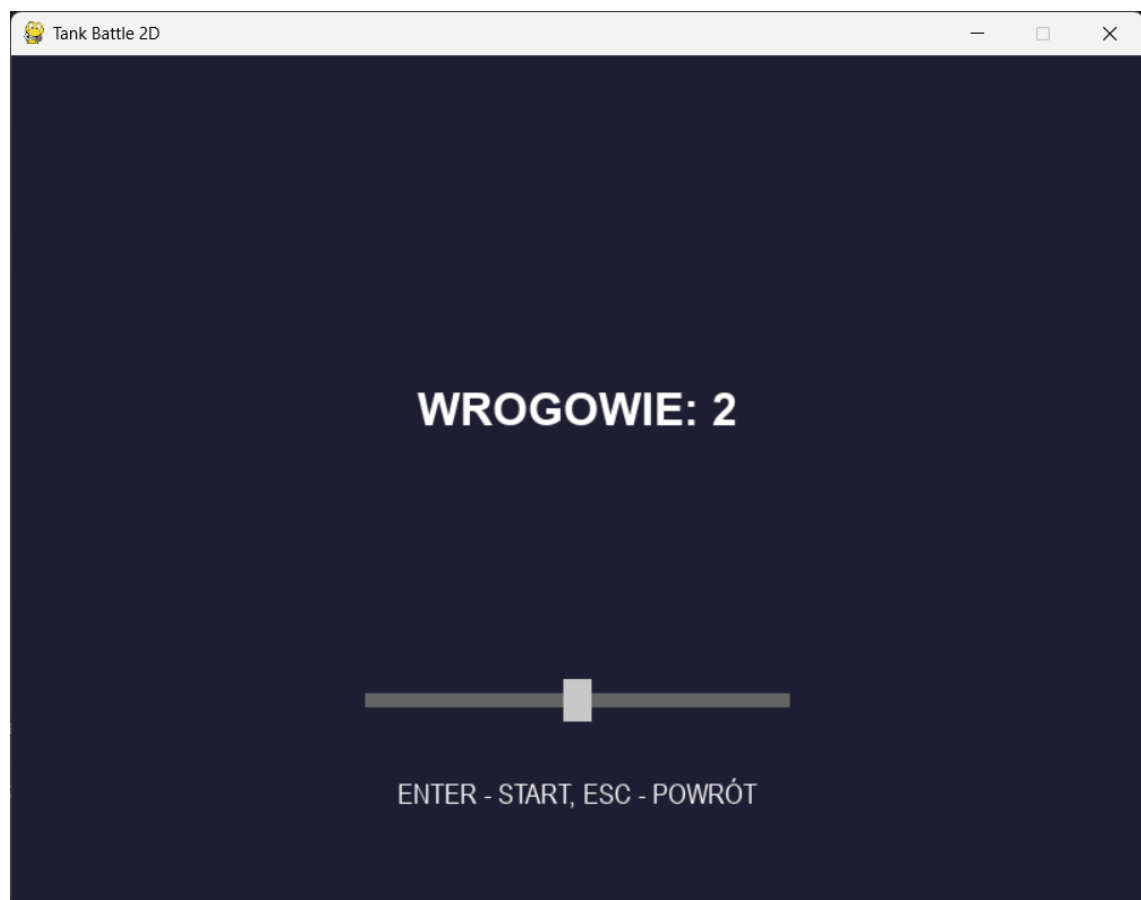
Rys. 3.1. Menu główne

3.2. Ekran rozgrywki - tryb Deathmatch

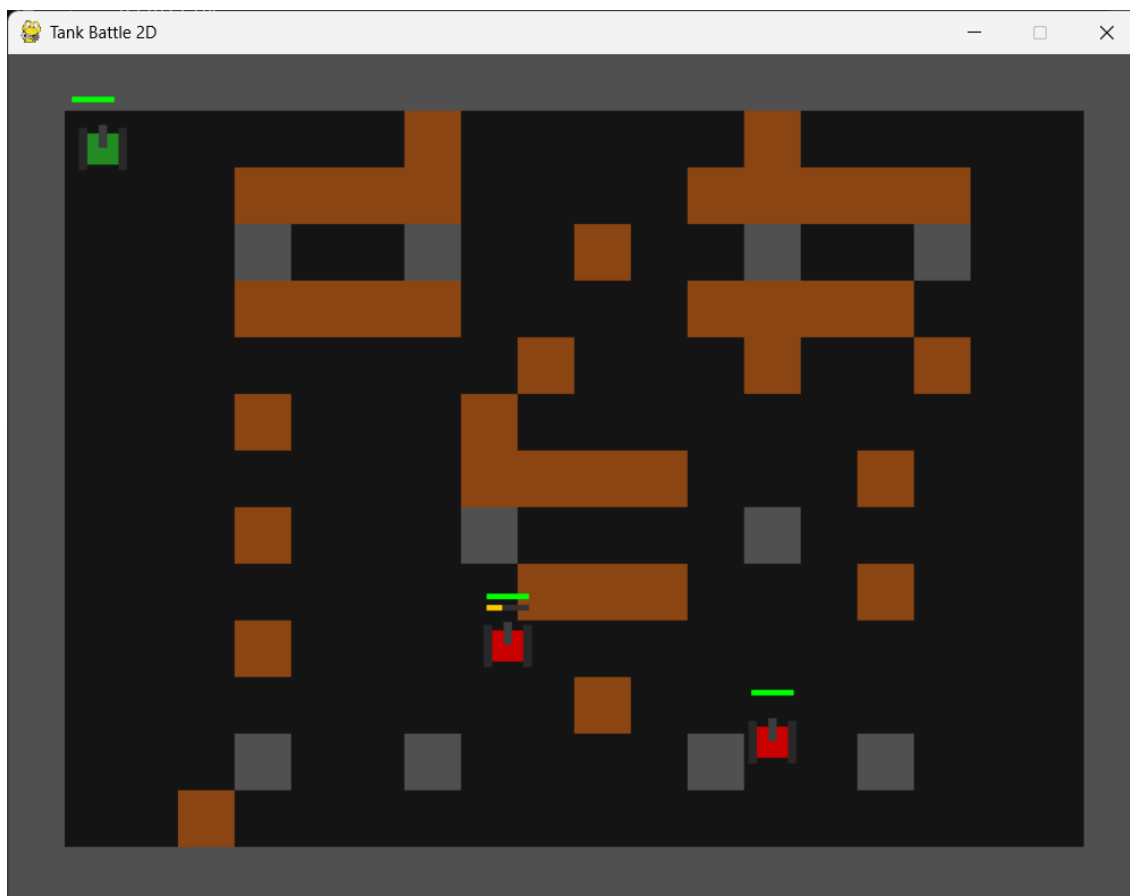
Ekran rozgrywki w trybie Deathmatch zawiera:

- Arenę 800x600 pikseli z losowo wybraną mapą z folderu Maps/
- Czołg gracza (kolor zielony, sterowanie strzałkami + spacja dostrzału)
- Od 1 do 3 czołgów przeciwnika sterowanych przez AI Minimax
- Pasek zdrowia nad każdym czołgiem (2 punkty zdrowia maks.)

- Ikony aktywnych power-upów
- Pojawiające się losowo power-upy (co 2,5 sekundy próba spawnu)



Rys. 3.2. Widok wyboru liczebności przeciwników zakres 1-3 (tryb Deathmatch)



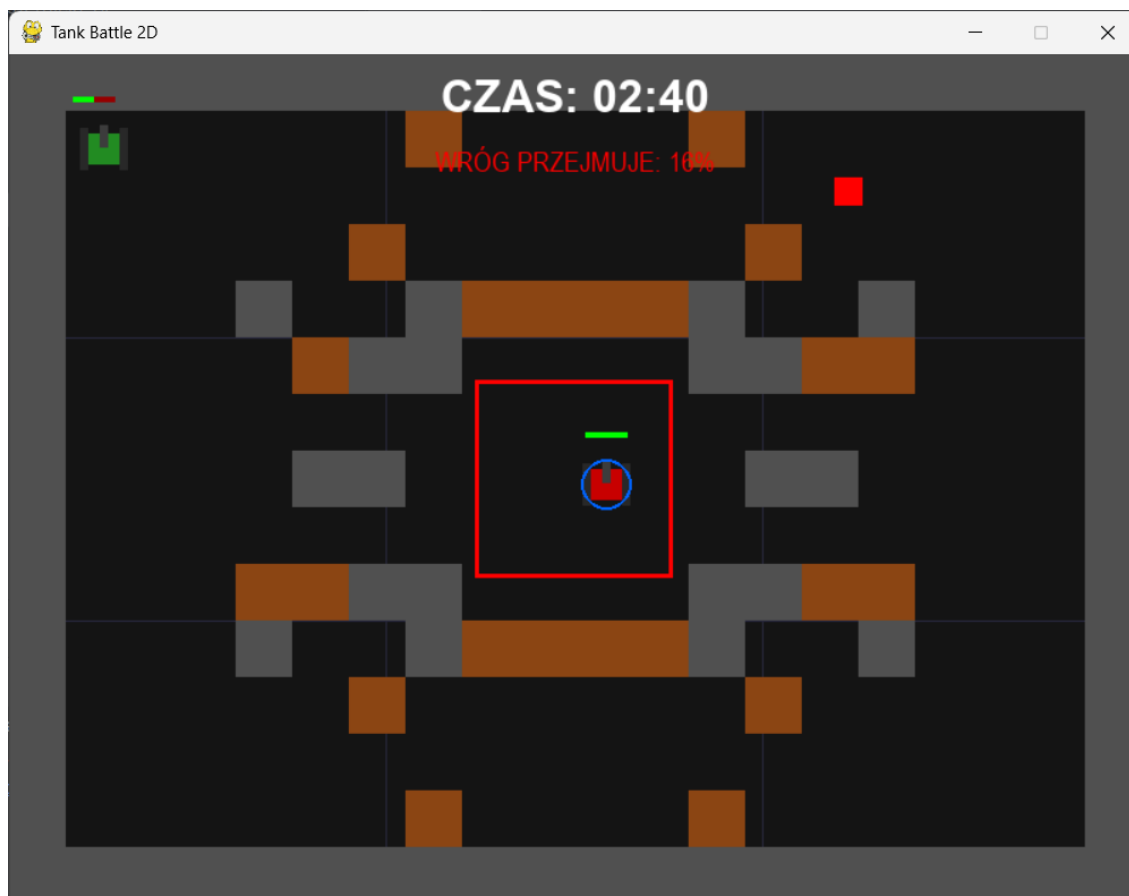
Rys. 3.3. Widok rozgrywki w trybie Deathmatch

Celem gracza jest wyeliminowanie wszystkich przeciwników. Możliwe wyniki rundy:

- *Wygrana* - gracz pokonał wszystkich przeciwników
- *Przegrana* - gracz został zniszczony
- *Remis* - gracz i ostatni przeciwnik zostają zniszczeni jednocześnie

3.3. Ekran rozgrywki - tryb Capture Point

W trybie Capture Point arena jest podzielona na siatkę 3x3 sektorów (9 sektorów). Na mapie wyświetlana jest strefa przejmowania (na starcie w sektorze centralnym), oraz pasek postępu przejmowania u góry ekranu.



Rys. 3.4. Widok rozgrywki w trybie Capture Point

Zasady przejmowania:

- Przebywanie gracza w strefie bez przeciwnika zwiększa postęp przejęcia
- Przebywanie przeciwnika w strefie bez gracza zmniejsza postęp
- Gdy obaj są w strefie, postęp się nie zmienia

Dynamika rozgrywki:

- Co 25 % przejęcia punkt przenosi się losowo do innego sektora, a po przeciwnej stronie mapy pojawia się nowy przeciwnik
- Co 15 % postępu gracza pojawia się dodatkowy przeciwnik
- Aby wygrać należy przejąć punkt do 100%. Jeśli wróg przejmie punkt do -100% - gracz przegrywa
- Rozgrywka trwa 2 minuty 50 sekund - jeśli w tym czasie nikt nie osiągnie 100%, runda kończy się jako „koniec czasu”

3.4. System power-upów

Power upy to bonusy pojawiające się losowo na mapie w wolnych miejscach (poza ścianami). Każdy power-up znika automatycznie po 15 sekundach, jeśli nikt go nie zbierze. Na mapie mogą jednocześnie występować maksymalnie 2 power-upy.

Zielony - Życie (life):

- Dla gracza w Deathmatch: ustawia licznik dodatkowych żyć na 1, gracz wskrzeszany jest po śmierci ze zdrowiem 2/2 w pozycji startowej
- Dla gracza w Capture Point: zwiększa licznik dodatkowych żyć o 1 i leczy gracza do pełnego HP
- Dla wroga (oba tryby): jeśli wróg jest ranny - leczy go do pełnego HP

Niebieski - Pancerz

Pochłania jedno trafienie pociskiem albo jedną kolizję z innym czołgiem, poczym znika. Efekt graficzny: niebieskie obramowanie wokół czołgu.

Czerwony - Silny strzał

Następny wystrzelony pocisk zadaje 2 obrażenia zamiast 1, co wystarczy do zniszczenia dowolnego czołgu jednym strzałem. Efekt znika natychmiast po oddaniu strzału. Efekt graficzny: czerwone obramowanie wokół czołgu i czerwono-pomarańczowy kolor pocisku.

3.5. Przykładowe scenariusze rozgrywki

W ramach demonstracji zachowania AI zarejestrowano 2 scenariusze rozgrywki, dostępne jako materiał wideo na platformie YouTube. Pierwszy przedstawia grę (zachowanie przeciwników) w trybie Deathmatch, drugi przedstawia grę (zachowanie przeciwników) w trybie Capture Point.

Rozgrywki w trybie Deathmatch:

Film : <https://www.youtube.com/watch?v=3lvFnlsOaLk>

Rozgrywka w trybie Capture point:

Film: <https://www.youtube.com/watch?v=EoK7oQob9l0>

3.6. Instrukcja uruchomienia aplikacji

1. Uruchomić środowisko PyCharm.
2. Sklonować repozytorium projektu z serwisu GitHub poleceniem: `git clone https://github.com/lukaszwarzocha/Projekt_SI.git`
3. Pobrać bibliotekę pygame poleceniem `pip install pygame` w terminalu wbudowanym w PyCharm lub przez interfejs zarządzania pakietami.
4. Uruchomić plik `main.py` (prawym przyciskiem myszy na pliku potem Run 'main', lub skrótem Shift+F10).

4. Podsumowanie

4.1. Wnioski z realizacji projektu

W ramach projektu zaimplementowano dwa klasyczne algorytmy sztucznej inteligencji w kontekście praktycznym - jako logikę decyzyjną przeciwników w grze komputerowej czasu rzeczywistego. Oba algorytmy działają poprawnie, generują wiarygodne zachowanie i stanowią mierzalne wyzwanie dla gracza.

Co udało się osiągnąć:

- Implementację algorytmu Minimax z przycinaniem alfa-beta na głębokość 4 z zaadaptowaną funkcją oceny pozycji uwzględniającą cztery komponenty (Czebyszew, LOS, power-up, koszt drogi)
- Implementację algorytmu A* z heurystyką euklidesową i różnicowymi wagami przejść kafelków
- Rozwiązanie kilku nietrywialnych problemów inżynierskich (snap do kafelka po rotacji 34x34, unikanie friendly fire, unik od sojusznika, optymalizacja THINK_EVERY)
- Bogaty system power-upów wpływający zarówno na gracza, jak i na bota
- Dwa odmienne tryby rozgrywki o różnych celach, prezentujące dwie odmienne klasy algorytmów SI

Wnioski algorytmiczne:

- Algorytm Minimax z przycinaniem alfa-beta sprawdza się w grze czasu rzeczywistego nawet przy głębokości 4, pod warunkiem cyklicznego uruchamiania (THINK_EVERY=15). Pełne przeszukiwanie na każdą klatkę nie jest konieczne ani efektywne.
- Wybór heurystyki jest kluczowy. Wartość -2 przy odległości Czebyszewa, +80 za LOS, oraz -60 za symulowany strzał gracza - te konkretne liczby powstały po serii testów, w których obserwowano zachowanie bota i regulowano wagi. Wartości zbyt agresywne sprawiały, że bot nieostrożnie rzucał się na gracza; zbyt defensywne sprawiały, że bot uciekał gdy mógł zaatakować.
- A* z heurystyką euklidesową, mimo iż teoretycznie mniej informatywny niż Manhattan dla siatki 4-kierunkowej, działa poprawnie i efektywnie w małej siatce 20x15.

4.2. Możliwości rozszerzenia projektu

Projekt w obecnej formie stanowi kompletny prototyp, jednak istnieje szereg kierunków, w których można go rozwinąć:

- **Multiplayer sieciowy** - gra dwuosobowa przez sieć, z zachowaniem AI jako trzeciego/czwartego gracza.
- **Reinforcement Learning** - zastąpienie heurystycznej funkcji oceny modelem uczonym. Pozwoliłoby to na AI nauczone "doświadczeniem" grania, niezależne od ręcznie dobranych wag.
- **Większe i bardziej zróżnicowane mapy** - obecny rozmiar 20x15 kafelków jest stosunkowo mały. Większe mapy uwidatniłyby różnice między algorytmami i potencjalnie wymagałyby głębszego Minimax.

- **Statystyki rozgrywki** - śledzenie wyników, najlepszych czasów, eliminacji - pozwoliłoby na bardziej ilościową ocenę skuteczności AI.
- **Wiele typów wrogów** - różne klasy botów z różnymi parametrami (szybkością, częstością strzału, agresywnością) lub różnymi algorytmami decyzyjnymi.

Spis rysunków

3.1	Menu główne	16
3.2	Widok wyboru liczebności przeciwników zakres 1-3 (tryb Deathmatch)	17
3.3	Widok rozgrywki w trybie Deathmatch	18
3.4	Widok rozgrywki w trybie Capture Point	19

Spis tabel

2.1	Wagi przejść kafelków w algorytmie A^*	12
-----	--	----

Spis listingów

2.1	Implementacja Minimax z przycinaniem alfa-beta (fragment z enemy.py)	10
2.2	Implementacja algorytmu A* (fragment z enemy2.py)	12